*Authored by: Arpit Raj, Lnmiit jaipur*

# What is Denormalization?

## Definition

Denormalization is the process of intentionally adding redundancy to a normalized database to improve query performance, especially for read-heavy workloads.

## Example

### Normalized Design

| Student | Department |
| --- | --- |
| StudentID | DepartmentID |
| Name | DepartmentName |
| DepartmentID | HOD |

To display a student with their department:

```
SELECT Student.Name, Department.DepartmentName
FROM Student
JOIN Department
ON Student.DepartmentID = Department.DepartmentID;
```

Every query requires a JOIN.

### Denormalized Design

| Student |
| --- |
| StudentID |
| Name |
| DepartmentID |
| DepartmentName |

Now:

```
SELECT Name, DepartmentName
FROM Student;
```

No JOIN needed.
Much faster for reading.

## JOIN operations become expensive as databases grow.

Large companies often have:

- Billions of rows
- Hundreds of tables
- Thousands of queries per second

Sometimes performing multiple JOINs becomes slower than storing duplicate data.

> Suppose every time Aadya opens the college portal, the application executes:
>
> Student
> JOIN Department
> JOIN Faculty
> JOIN Semester
> JOIN Classroom
> JOIN Attendance
> JOIN Grades

Six joins for every request.
Millions of users.
Very expensive.
Instead,
the system may store frequently accessed information together.

## Read-Heavy Systems

Some systems perform:

- 99% reads
- 1% writes

Examples

- News websites
- Netflix homepage
- Amazon product pages.

These benefit greatly from denormalization.

## Write-Heavy Systems

Some systems perform frequent updates.

Examples

- Banking
- Payment systems
- Airline booking
- Inventory management

These usually remain normalized because duplicate data makes updates difficult.

## Materialized Views

### Definition

A Materialized View stores the result of a query physically.
Unlike a normal view,
the result is already computed.

### Example

**Normal View**

```
CREATE VIEW StudentGrades AS
SELECT ...
```

Every query recomputes the result.

**Materialized View**

Stores the result.

| StudentID | Name | AverageMarks | CGPA |
|---|---|---|---|

No computation needed during reads.

Used in

- Analytics
- Dashboards
- Reporting

## Data Warehouses

Data warehouses are mostly:

- Read-only.
- Historical.
- Huge.

Denormalization is extremely common.

### Example

**Sales Warehouse**
FactSales
Already contains

- Product Name
- Category
- Region

- Customer Segment

No joins needed.

## 11. Analytics

Analytics systems execute
Millions of aggregate queries.

Example
Average GPA of all ECE students.

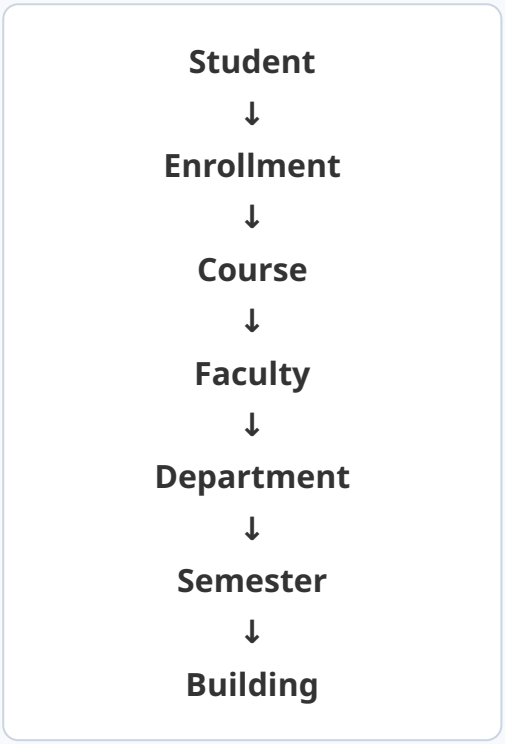
If every query performs joins,
analytics become slow.

## 12. Dashboards

Management dashboards refresh every few seconds.
Instead of
joining 20 tables,
store summarized information.

## 13. Large Joins

Suppose




One page requires six joins.
Companies often replace this with
one summary table.

## 14. Caching

Frequently requested data is duplicated into
Cache.

Example
Student Profile Cache
Stores
Everything required
without joins.

## 5. OLAP Systems

OLAP
Online Analytical Processing
Uses

- Fact tables
- Dimension tables

Often denormalized.
Star Schema is a famous example.

## Advantages of Denormalization

1. **Faster Reads**
   Fewer joins. Lower execution time.
2. **Fewer JOIN Operations**
   Most queries become simpler.
3. **Better Reporting**
   Reports execute much faster.
4. **Simpler Queries**
   Instead of JOIN JOIN JOIN JOIN
   Simple SELECT.
5. **Lower Query Latency**
   Very important for Web apps, Dashboards, Mobile apps

## Disadvantages of Denormalization

1. **Data Redundancy**
   Duplicate data everywhere.
2. **Update Anomalies**
   Suppose the ECE department changes its name.
   Normalized: Update once.
   Denormalized: Update thousands of rows.
3. **Storage Overhead**
   Duplicate information consumes more disk space.
4. **Inconsistent Data**
   If one copy updates but another doesn't, inconsistencies appear.
   Example: Student table (ECE), Report table (Electronics). Which is correct?
5. **Complex Updates**
   Insert and Update operations become harder because multiple copies of the same information must remain synchronized.
6. **Integrity Maintenance**
   Maintaining data integrity becomes more difficult.
   Applications or database triggers may be needed to ensure all duplicate copies stay consistent.

## Can OLTP systems use denormalization?

**Answer**

Yes, but carefully.

Most OLTP systems are primarily normalized (often up to 3NF). However, they may selectively denormalize specific tables or add precomputed columns, summary tables, or caches for performance-critical queries.